



Certified Kubernetes Administrator (CKA)

By Cloud Mentor Pro

Updated: 10/04/2026



Table of Contents

- Section 21 - Mock Exam - 1
 - *Mock Exam - 1*
 - *Mock Exam - 2 (Level Up!)*
 - *Mock Exam - 3 (Wohooo! Level Up!)*

Mock Exam - 1

- *12 Questions*

Q1

1 2 3 4 5 6 7 8 9 10 11 12



Weight: 6

Deploy a pod named `nginx-pod` using the `nginx:alpine` image.

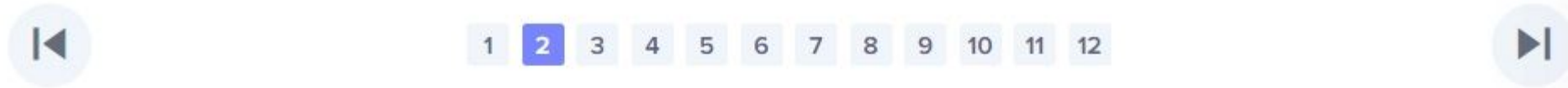
Once done, click on the **Next Question** button in the top right corner of this panel. You may navigate back and forth freely between all questions. Once done with all questions, click on **End Exam**. Your work will be validated at the end and score shown. Good Luck!

- Name: `nginx-pod`
- Image: `nginx:alpine`

Q1 - Answer

```
k run nginx-pod --image=nginx:alpine
```

Q2



Weight: 8

Deploy a `messaging` pod using the `redis:alpine` image with the labels set to `tier=msg`.

- Pod Name: messaging
- Image: redis:alpine
- Labels: tier=msg

Q2 - Answer

```
controlplane ~ → kgp --show-labels
NAME      READY   STATUS    RESTARTS   AGE   LABELS
messaging 1/1     Running   0           10s   tier=msg
nginx-pod 1/1     Running   0           110s  run=nginx-pod
```

```
k run messaging --image=redis:alpine --labels='tier=msg'
```

Q3



1 2 3 4 5 6 7 8 9 10 11 12

Weight: 4

Create a namespace named `apx-x9984574` .

● Namespace: apx-x9984574

Q3 - Answer

```
k create ns apx-x9984574
```

Q4



Weight: 7

Get the list of nodes in JSON format and store it in a file at `/opt/outputs/nodes-z3444kd9.json`.

● Task completed

Q4 - Answer

```
k get node -o json > /opt/outputs/nodes-z3444kd9.json  
cat /opt/outputs/nodes-z3444kd9.json
```

Q5



1 2 3 4 5 6 7 8 9 10 11 12



Weight: 12

Create a service `messaging-service` to expose the `messaging` application within the cluster on port `6379`.

Use imperative commands.

- Service: `messaging-service`
- Port: `6379`
- Type: `ClusterIp`
- Use the right labels

Q5 - Answer

```
Terminal 1  Terminal 2  +
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    tier: msg
  name: messaging-service
spec:
  ports:
    - port: 6379
      protocol: TCP
      targetPort: 6379
  selector:
    tier: msg
  type: ClusterIP
status:
  loadBalancer: {}
```

```
controlplane ~ → kgs
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
kubernetes           ClusterIP   10.96.0.1     <none>         443/TCP    4h41m
messaging-service    ClusterIP   10.98.255.192 <none>         6379/TCP    7s

controlplane ~ → k describe svc messaging-service
Name:                messaging-service
Namespace:           default
Labels:              tier=msg
Annotations:         <none>
Selector:            tier=msg
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.98.255.192
IPs:                 10.98.255.192
Port:                <unset> 6379/TCP
TargetPort:          6379/TCP
Endpoints:           10.244.0.5:6379
Session Affinity:    None
Events:              <none>

controlplane ~ →
```

```
k expose po messaging --name=messaging-service --port=6379 --target-port=6379 --type='ClusterIP'
--dry-run=client -o yaml > 5.yaml
```

Q6



1 2 3 4 5 6 7 8 9 10 11 12



Weight: 11

Create a deployment named `hr-web-app` using the image `kodekloud/webapp-color` with `2` replicas.

- Name: `hr-web-app`
- Image: `kodekloud/webapp-color`
- Replicas: `2`

Q6 - Answer

```
Terminal 1 Terminal 2 +
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: hr-web-app
  name: hr-web-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hr-web-app
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: hr-web-app
    spec:
      containers:
      - image: kodekloud/webapp-color
        name: webapp-color
        resources: {}
  status: {}
```

k create deployment hr-web-app --image=kodekloud/webapp-color --replicas=2 --dry-run=client -o yaml > 6.yaml

ka 6.yaml

Q7



1 2 3 4 5 6 7 8 9 10 11 12



Weight: 8

Create a static pod named `static-busybox` on the controlplane node that uses the `busybox` image and the command `sleep 1000`.

- Name: static-busybox
- Image: busybox

Q7 - Answer

```
controlplane ~ ❌ ls /etc/kubernetes/manifests/
etcd.yaml kube-apiserver.yaml kube-controller-manager.yaml kube-scheduler.yaml

controlplane ~ ➔ cp 7.yaml /etc/kubernetes/manifests/

controlplane ~ ➔ ls /etc/kubernetes/manifests/
7.yaml etcd.yaml kube-apiserver.yaml kube-controller-manager.yaml kube-scheduler.yaml
```

```
controlplane ~ ❌ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
hr-web-app-57cd7b5799-dwktk        1/1     Running   0           8m36s  10.244.0.6    controlplane
hr-web-app-57cd7b5799-1l8b6        1/1     Running   0           8m36s  10.244.0.7    controlplane
messaging                          1/1     Running   0           21m    10.244.0.5    controlplane
nginx-pod                          1/1     Running   0           23m    10.244.0.4    controlplane
static-busybox-controlplane        1/1     Running   6 (86s ago) 4m36s    10.244.0.8    controlplane
```

```
k run static-busybox --image=busybox --command sleep 1000 --dry-run=client -o yaml > 7.yaml
```

```
cp 7.yaml /etc/kubernetes/manifests/
```

Q7 - Bonus killser.sh

< Question 19 >

Task weight: 3%

NOTE: This task can only be solved if questions 18 or 20 have been successfully implemented and the k8s-c3-CCC cluster has a functioning worker node

Use context: `kubect1 config use-context k8s-c3-CCC`

Do the following in a new *Namespace* `secret` . Create a *Pod* named `secret-pod` of image `busybox:1.31.1` which should keep running for some time.

There is an existing Secret located at `/opt/course/19/secret1.yaml` , create it in the *Namespace* `secret` and mount it readonly into the *Pod* at `/tmp/secret1` .

Create a new Secret in *Namespace* `secret` called `secret2` which should contain `user=user1` and `pass=1234` . These entries should be available inside the *Pod*'s container as environment variables `APP_USER` and `APP_PASS`.

Confirm everything is working.

```
kr busybox:1.11>11 secret-pod - sh -c 'sleep 1d' | ka -
```

#Vì đề y.cầu “which should keep running for some time”, mà kq của mình có trong 24h

Q8



1 2 3 4 5 6 7 8 9 10 11 12



Weight: 12

Create a POD in the **finance** namespace named **temp-bus** with the image **redis:alpine**.

- Name: temp-bus
- Image Name: redis:alpine

Q8 - Answer

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: temp-bus
  name: temp-bus
  namespace: finance
spec:
  containers:
  - image: redis:alpine
    name: temp-bus
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

```
controlplane ~ → ka 8.yaml
pod/temp-bus created

controlplane ~ → kgp -n finance
NAME      READY   STATUS    RESTARTS   AGE
temp-bus  1/1     Running   0           5s
```

k run temp-bus --image=redis:alpine --dry-run=client -o yaml > 8.yaml

#Edit yaml thêm "namespace: finance"

Q9



1 2 3 4 5 6 7 8 9 10 11 12



Weight: 8

A new application **orange** is deployed. There is something wrong with it. Identify and fix the issue.

● Issue fixed

Q9 - Answer

```
controlplane ~ → kgp orange
NAME      READY   STATUS              RESTARTS   AGE
orange    0/1     Init:CrashLoopBackOff 2 (23s ago) 43s
```

```
Init Containers:
  init-myservice:
    Container ID:   containerd://f0a29c059f63ad1b8bed15328ed8e5f
    Image:          busybox
    Image ID:       docker.io/library/busybox@sha256:1ceb872bcc6
    Port:           <none>
    Host Port:      <none>
    Command:
      sh
      -c
      sleeep 2;
    State:          Waiting
    Reason:          CrashLoopBackOff
    Last State:     Terminated
    Reason:          Error
    Exit Code:      127

controlplane ~ ✖ k logs orange -c init-myservice
sh: sleeep: not found
```

```
controlplane ~ → kgp -w
NAME                                READY   STATUS    RESTARTS   AGE
hr-web-app-57cd7b5799-dwktk        1/1     Running   0           24m
hr-web-app-57cd7b5799-1l8b6        1/1     Running   0           24m
messaging                          1/1     Running   0           37m
nginx-pod                          1/1     Running   0           39m
orange                             1/1     Running   0           10s
```

k describe po orange
Warning BackOff 1s (x7 over 74s) kubelet Back-off restarting failed container init-myservice in pod orange_default

k logs orange -c init-myservice #sh: sleeep: not found
=> SOLUTION: Correct command from "sleeep" to "sleep"
k edit po orange

Q10

◀

123456789101112

▶

Weight: 10

Expose the `hr-web-app` as service `hr-web-app-service` application on port `30082` on the nodes on the cluster.

The web application listens on port `8080` .

- Name: `hr-web-app-service`
- Type: `NodePort`
- Endpoints: 2
- Port: `8080`
- NodePort: `30082`

Q10 - Answer

```
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    app: hr-web-app
    name: hr-web-app-service
spec:
  ports:
    - port: 8080
      protocol: TCP
      targetPort: 8080
      nodePort: 30082
  selector:
    app: hr-web-app
  type: NodePort
status:
  loadBalancer: {}
```

```
controlplane ~ → ka 10.yaml
service/hr-web-app-service created
```

```
controlplane ~ → kgs
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
hr-web-app-service	NodePort	10.103.184.77	<none>	8080:30082/TCP	5s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	5h19m
messaging-service	ClusterIP	10.98.255.192	<none>	6379/TCP	38m

```
controlplane ~ → k describe svc hr-web-app-service
```

Name: hr-web-app-service
Namespace: default
Labels: app=hr-web-app
Annotations: <none>
Selector: app=hr-web-app
Type: NodePort
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.103.184.77
IPs: 10.103.184.77
Port: <unset> 8080/TCP
TargetPort: 8080/TCP
NodePort: <unset> 30082/TCP
Endpoints: 10.244.0.6:8080,10.244.0.7:8080
Session Affinity: None
External Traffic Policy: Cluster
Events: <none>

```
k expose deployment hr-web-app --name=hr-web-app-service --port=8080 --target-port=8080 --type='NodePort' --dry-run=client -o yaml > 10.yaml
```

```
vi 10.yaml #Them "nodePort: 30082"
```


Q11



1

2

3

4

5

6

7

8

9

10

11

12



Weight: 6

Use JSON PATH query to retrieve the `osImage` s of all the nodes and store it in a file `/opt/outputs/nodes_os_x43kj56.txt` .

The `osImages` are under the `nodeInfo` section under `status` of each node.

● Task Completed

Q11 - Answer

```
k get nodes -o=jsonpath='{.items[*].status.nodeInfo.osImage}' > /opt/outputs/nodes_os_x43kj56.txt  
cat /opt/outputs/nodes_os_x43kj56.txt #Ubuntu 20.04.6 LTS
```

Q12



1 2 3 4 5 6 7 8 9 10 11 12

Weight: 8

Create a **Persistent Volume** with the given specification: -

Volume name: **pv-analytics**

Storage: **100Mi**

Access mode: **ReadWriteMany**

Host path: **/pv/data-analytics**

- ☐ Is the volume name set?
- ☐ Is the storage capacity set?
- ☐ Is the accessMode set?
- ☐ Is the hostPath set?

Q12 - Answer

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-analytics
spec:
  capacity:
    storage: 100Mi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/pv/data-analytics"
```

```
controlplane ~ → vi 12.yaml
```

```
controlplane ~ → ka 12.yaml
```

```
persistentvolume/pv-analytics created
```

```
controlplane ~ → k get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	REASON	AGE
pv-analytics	100Mi	RWX	Retain	Available				7s

```
vi 12.yaml
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-analytics
spec:
  capacity:
    storage: 100Mi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/pv/data-analytics"
```

Mock Exam - 2 (Level Up!)

- *12 Questions*

Mock Exam - 3 (Wohoooo! Level Up!)

- *9 Questions*